

David_字符串

David_字符串

- 字典树
- AC自动机
- KMP与马拉车
- 字符串哈希
- 后缀数组
- 后缀自动机
- 后缀树

字典树

```
1  const int N=3e6+10;
2  const int base=13331;
3  const int mod=1e9+7;
4  int n,_T,q;
5  string s;
6  int ch[N][70]={0};
7  int idx;
8  int cnt[N];
9
10 int hashn(char x)
11 {
12     if(x>='A'&&x<='Z')
13     {
14         return x-'A';
15     }
16     else if(x>='a'&&x<='z')
17     {
18         return x-'a'+26;
19     }
20     else
21         return x-'0'+52;
22 }
23
24 void insert(string s)
25 {
26     int p=0;
27     int len=s.length();
28     for(int i=0;i<len;i++)
29     {
30         int j=hashn(s[i]);
31         if(!ch[p][j]) ch[p][j]=++idx;
32         p=ch[p][j];
33         cnt[p]++;
34     }
35 }
36
37
38 int query(string s)
39 {
40     int p=0;
```

```

41     int len=s.length();
42     for(int i=0;i<len;i++)
43     {
44         int j=hashn(s[i]);
45         if(!ch[p][j]) return 0;
46         p=ch[p][j];
47     }
48     return cnt[p];
49 }
50
51 void solve()
52 {
53     idx=0;
54     cin>>n>>q;
55     for(int i=1;i<=n;i++)
56     {
57         cin>>s;
58         insert(s);
59     }
60     while(q--)
61     {
62         cin>>s;
63         cout<<query(s)<<endl;
64     }
65     for(int i=0;i<=idx;i++)
66     {
67         for(int j=0;j<=65;j++)
68         {
69             ch[i][j]=0;
70         }
71     }
72     for(int i=0;i<=idx;i++)
73     {
74         cnt[i]=0;
75     }
76 }

```

AC自动机

```

1 //AC自动机模板
2 // 结构: Trie + fail 指针; 文本在自动机上跑, 状态 u = 当前文本前缀的最长后缀
3 // 关键: fail[u] 指向"u 代表串的最长真后缀所在状态", build 时补齐 son 转移, 失配自动跳 fail
4 //
5 //AC自动机常用来解决什么:
6 // - 多模式串在文本中的出现次数 / 出现位置 (洛谷 P3808 / P3796 模板题)
7 // - 文本是否包含任一模式串、每个模式串首次出现位置 (见 query_first)
8 // - 屏蔽词类: 每个文本位置结尾的最长匹配模式串 -> 区间覆盖 DP (见 covered)
9 // - 多模式串 + 自动机 DP (状态含 fail 链的转移)
10 //判别信号: 模式串数量多(几十~几百个), 需对同一文本批量查询; 模式串总长 1e6 级别
11 //适用注意: 建好后模式串集合不再变; 模式串会动态变化时不适合(重建太贵)
12 struct AC {
13     static const int SIZE = 2e5 + 6; // 模式串总长度上界
14
15     struct Node {
16         int son[26]; // 转移边 (build 后填满, 未匹配字符自动跳 fail)
17         int len; // 该节点在 Trie 中的深度 = 代表串长度 (应用1/2 需要)
18         int ans; // 文本跑到该状态的次数 (query 时累加)
19         int fail; // 失败指针
20         int du; // fail 树入度, 用于拓扑传播计数
21         int idx; // 模式串编号 (相同模式串映射到同一编号)

```

```

22
23     void init() {
24         memset(son, 0, sizeof(son));
25         len = ans = fail = idx = 0;
26     }
27 };
28
29 Node tr[SIZE];
30 int tot;           // 节点总数
31 int ans[SIZE];     // 每个模式串的最终出现次数
32 int pidx;          // 模式串编号计数器
33 vector<int> qorder; // build 的 BFS 顺序 (按 Trie 深度递增, 应用1 用)
34 int maxLen[SIZE];  // 每个状态结尾能匹配到的最长模式串长度 (应用1 用)
35
36 void init() {
37     tot = pidx = 0;
38     qorder.clear();
39     memset(ans, 0, sizeof(ans));
40     memset(maxLen, 0, sizeof(maxLen));
41     tr[0].init();
42 }
43
44 // 参数: 模式串 s (0-base), id 引用返回该模式串的编号
45 void insert(const string& s, int& id) {
46     int u = 0;
47     for (int i = 0; i < (int)s.size(); i++) {
48         int c = s[i] - 'a';
49         if (!tr[u].son[c]) {
50             tr[u].son[c] = ++tot;
51             tr[tot].init();
52         }
53         u = tr[u].son[c];
54         tr[u].len = i + 1;
55     }
56     if (!tr[u].idx) tr[u].idx = ++pidx;
57     id = tr[u].idx;
58 }
59
60 void build() {
61     queue<int> q;
62     for (int i = 0; i < 26; i++) {
63         if (tr[0].son[i]) q.push(tr[0].son[i]);
64     }
65     while (!q.empty()) {
66         int u = q.front();
67         q.pop();
68         qorder.push_back(u);
69         for (int i = 0; i < 26; i++) {
70             int v = tr[u].son[i];
71             if (v) {
72                 tr[v].fail = tr[tr[u].fail].son[i];
73                 tr[tr[v].fail].du++; // fail 树入度
74                 q.push(v);
75             } else {
76                 tr[u].son[i] = tr[tr[u].fail].son[i]; // 补齐转移, 查询时无需判
失配
77             }
78         }
79     }
80 }
81

```

```

82 // 跑一遍文本，每个状态累加"文本前缀以该状态结尾"的次数
83 void query(const string& t) {
84     int u = 0;
85     for (char ch : t) {
86         u = tr[u].son[ch - 'a'];
87         tr[u].ans++;
88     }
89 }
90
91 // 拓扑传播计数：每个模式串出现次数 = 其 fail 子树内所有状态的 ans 之和
92 // (每个文本位置都可能在 fail 链上代表多个模式串，故必须累加)
93 void topu() {
94     queue<int> q;
95     for (int i = 0; i <= tot; i++) {
96         if (tr[i].du == 0) q.push(i);
97     }
98     while (!q.empty()) {
99         int u = q.front();
100        q.pop();
101        if (tr[u].idx) ans[tr[u].idx] = tr[u].ans;
102        int v = tr[u].fail;
103        tr[v].ans += tr[u].ans;
104        if (--tr[v].du) q.push(v);
105    }
106 }
107
108 // 应用1：每个状态结尾能匹配到的最长模式串长度
109 // 正确性：maxLen[u] = max(maxLen[fail[u]], 以 u 结尾的模式串长度)
110 // 沿 fail 链取 max 即"当前文本前缀的所有后缀里最长的模式串"
111 void compute_maxlen() {
112     memset(maxLen, 0, sizeof(maxLen));
113     for (int u : qorder) { // BFS 序 = 深度递增，fail 必已算过
114         maxLen[u] = maxLen[tr[u].fail];
115         if (tr[u].idx) maxLen[u] = max(maxLen[u], tr[u].len);
116     }
117 }
118
119 // 应用2：每个模式串首次出现的起始位置 (0-base, 未出现为 -1)
120 // 复杂度警告：每步沿 fail 链走，最坏 O(文本长 × 最长模式串长)；大数据需优化为 fail 树
+倍增
121 vector<int> first;
122 void query_first(const string& t) {
123     first.assign(pidx + 1, -1);
124     int u = 0;
125     for (int i = 0; i < (int)t.size(); i++) {
126         u = tr[u].son[t[i] - 'a'];
127         for (int x = u; x; x = tr[x].fail) { // fail 祖先都是"以 i 结尾的模式
串"
128             if (tr[x].idx && first[tr[x].idx] == -1) {
129                 first[tr[x].idx] = i - tr[x].len + 1; // 记录起始位置
130             }
131         }
132     }
133 }
134
135 // 应用3：文本是否被模式串完全覆盖
136 // 思路：每个位置 i 用"最长匹配长度 L"给出覆盖区间 [i-L+1, i];
137 // 较短匹配的区间是它的子集，所以合并这些区间即可判定全覆盖
138 bool covered(const string& t) {
139     compute_maxlen();
140     vector<pair<int, int>> segs;

```

```

141     int u = 0;
142     for (int i = 0; i < (int)t.size(); i++) {
143         u = tr[u].son[t[i] - 'a'];
144         if (maxLen[u]) segs.push_back({i - maxLen[u] + 1, i});
145     }
146     sort(segs.begin(), segs.end());
147     int cur = 0; // 已覆盖 [0, cur)
148     for (auto [l, r] : segs) {
149         if (l > cur) return false; // 出现断档
150         cur = max(cur, r + 1);
151         if (cur >= (int)t.size()) return true;
152     }
153     return cur >= (int)t.size();
154 }
155 } ac;

```

KMP与马拉车

```

1  const int N=1e6+10;
2  int n,m;
3  char s1[N],s2[N];
4  string s,p;
5  int ne[N];
6
7  //马拉车求最长回文串
8  int d[N];
9  void getd()
10 {
11     d[1]=1;
12     for(int i=2,l=0,r=1;i<=n;i++)
13     {
14         if(i<=r) d[i]=min(r-i+1,d[r-i+1]);
15         while(s[i-d[i]]==s[i+d[i]]) d[i]++;
16         if(i+d[i]-1>r) l=i-d[i]+1,r=i+d[i]-1;
17     }
18 }
19
20 //KMP预处理next数组
21 ne[1]=0;
22 for(int i=2;i<=n;i++)
23 {
24     while(j&&s2[i]!=s2[j+1]) j=ne[j];
25     if(s2[i]==s2[j+1]) j++;
26     ne[i]=j;
27 }
28 //KMP匹配
29 for(int i=1,j=0;i<=m;i++)
30 {
31     while(j&&s1[i]!=s2[j+1]) j=ne[j];
32     if(s1[i]==s2[j+1]) j++;
33     if(j==n)
34     {
35         cout<<i-n+1<<endl;
36         j=ne[j];
37     }
38 }

```

字符串哈希

```

1 //字符串哈希
2 struct StringHash {
3     static const int MAXN = 1e6 + 5;
4     static const int BASE = 277;
5     static const int MOD1 = 1e9 + 7;
6     static const int MOD2 = 1e9 + 9;
7
8     inline static vector<int> p1, p2;
9     vector<int> h1, h2;
10    int n;
11
12    // 初始化静态成员数组 p1 和 p2
13    static void get_power() {
14        static bool initialized = false;
15        if (!initialized) {
16            p1.assign(MAXN, 0);
17            p2.assign(MAXN, 0);
18            p1[0] = 1;
19            p2[0] = 1;
20            for (int i = 1; i < MAXN; i++) {
21                p1[i] = p1[i - 1] * BASE % MOD1;
22                p2[i] = p2[i - 1] * BASE % MOD2;
23            }
24            initialized = true;
25        }
26    }
27
28    // 构造函数: 传入字符串, 自动计算前缀哈希
29    StringHash(const string& s) {
30        get_power();
31        n = s.size();
32        h1.assign(n + 1, 0);
33        h2.assign(n + 1, 0);
34        for (int i = 0; i < n; i++) {
35            h1[i + 1] = (h1[i] * BASE + s[i]) % MOD1;
36            h2[i + 1] = (h2[i] * BASE + s[i]) % MOD2;
37        }
38    }
39
40    // 获取子串 [l, r] 的双哈希值 (0-indexed)
41    // 例如 s = "abcde", get(1, 3) 返回 "bcd" 的哈希
42    pair<int, int> get(int l, int r) {
43        int len = r - l + 1;
44        int res1 = (h1[r + 1] - h1[l] * p1[len] % MOD1 + MOD1) % MOD1;
45        int res2 = (h2[r + 1] - h2[l] * p2[len] % MOD2 + MOD2) % MOD2;
46        return {res1, res2};
47    }
48 };
49
50 struct MatrixHash {
51     static const int MAXN = 1005; // 最大行数
52     static const int MAXM = 1005; // 最大列数
53     static const int BASE_R = 13331; // 行基数
54     static const int BASE_C = 277; // 列基数
55     static const int MOD1 = 1e9 + 7;
56     static const int MOD2 = 1e9 + 9;
57
58     // 静态成员数组: 行幂次与列幂次
59     inline static vector<int> pr1, pr2, pc1, pc2;
60
61     // 二维前缀哈希表

```

```

62     vector<vector<int>> h1, h2;
63     int R, C;
64
65     // 初始化静态幂次数组
66     static void get_power() {
67         static bool initialized = false;
68         if (!initialized) {
69             pr1.assign(MAXN, 0); pr2.assign(MAXN, 0);
70             pc1.assign(MAXM, 0); pc2.assign(MAXM, 0);
71             pr1[0] = pr2[0] = pc1[0] = pc2[0] = 1;
72
73             for (int i = 1; i < MAXN; i++) {
74                 pr1[i] = 1LL * pr1[i - 1] * BASE_R % MOD1;
75                 pr2[i] = 1LL * pr2[i - 1] * BASE_R % MOD2;
76             }
77             for (int i = 1; i < MAXM; i++) {
78                 pc1[i] = 1LL * pc1[i - 1] * BASE_C % MOD1;
79                 pc2[i] = 1LL * pc2[i - 1] * BASE_C % MOD2;
80             }
81             initialized = true;
82         }
83     }
84
85     // 注意传入数组是 字符串 还是 int !!!!
86     // 构造函数: 传入 vector<string> 或 vector<vector<int>>
87     MatrixHash(const vector<string>& mat) {
88         get_power();
89         R = mat.size();
90         C = mat[0].size();
91         h1.assign(R + 1, vector<int>(C + 1, 0));
92         h2.assign(R + 1, vector<int>(C + 1, 0));
93
94         for (int i = 0; i < R; i++) {
95             for (int j = 0; j < C; j++) {
96                 // 二维哈希递推公式:  $H[i][j] = H[i-1][j]*Br + H[i][j-1]*Bc - H[i-1][j-1]*Br*Bc + val$ 
97                 h1[i + 1][j + 1] = (1LL * h1[i][j + 1] * BASE_R % MOD1 +
98                                     1LL * h1[i + 1][j] * BASE_C % MOD1 -
99                                     1LL * h1[i][j] * BASE_R % MOD1 * BASE_C %
100                 MOD1 +
101                                     mat[i][j] + MOD1) % MOD1;
102
103                 h2[i + 1][j + 1] = (1LL * h2[i][j + 1] * BASE_R % MOD2 +
104                                     1LL * h2[i + 1][j] * BASE_C % MOD2 -
105                                     1LL * h2[i][j] * BASE_R % MOD2 * BASE_C %
106                 MOD2 +
107                                     mat[i][j] + MOD2) % MOD2;
108             }
109         }
110
111         // 获取子矩阵 [(r1, c1), (r2, c2)] 的双哈希值 (0-indexed) !!!
112         pair<int, int> get(int r1, int c1, int r2, int c2) {
113             int dr = r2 - r1 + 1; // 行高
114             int dc = c2 - c1 + 1; // 列宽
115
116             // 提取公式:  $H = h[r2][c2] - h[r1-1][c2]*Br^{dr} - h[r2][c1-1]*Bc^{dc} + h[r1-1][c1-1]*Br^{dr}*Bc^{dc}$ 
117             int res1 = (h1[r2 + 1][c2 + 1] - 1LL * h1[r1][c2 + 1] * pr1[dr] % MOD1 +
118                         MOD1) % MOD1;
119             res1 = (res1 - 1LL * h1[r2 + 1][c1] * pc1[dc] % MOD1 + MOD1) % MOD1;

```

```

118     res1 = (res1 + 1LL * h1[r1][c1] * pr1[dr] % MOD1 * pc1[dc] % MOD1) %
MOD1;
119
120     int res2 = (h2[r2 + 1][c2 + 1] - 1LL * h2[r1][c2 + 1] * pr2[dr] % MOD2 +
MOD2) % MOD2;
121     res2 = (res2 - 1LL * h2[r2 + 1][c1] * pc2[dc] % MOD2 + MOD2) % MOD2;
122     res2 = (res2 + 1LL * h2[r1][c1] * pr2[dr] % MOD2 * pc2[dc] % MOD2) %
MOD2;
123
124     return {res1, res2};
125 }
126 };

```

后缀数组

```

1 //后缀数组模板 (倍增 + 基数排序,  $O(n \log n)$ )
2 //sa[i] : 排名第 i 的后缀起点 (0-base)
3 //rnk[i] : 起点 i 的后缀排名
4 //height[i]: LCP(sa[i-1], sa[i]), height[0] = 0 (height 数组左闭右开地服务 RMQ)
5 //
6 //后缀数组常用来解决什么:
7 // - 任意两个后缀的 LCP (配合 RMQ, 见 lcp())
8 // - 子串计数: 不同子串数、某子串出现次数、本质不同子串第 k 小
9 // - 重复子串: 最长重复子串(可重叠=max height / 不可重叠=二分+分组)、出现次数>=K 的最长子串
10 // - 多串问题: 两个/多个串的最长公共子串 (连接后扫相邻 height)
11 // - 循环节、最小循环表示
12 //判别信号: 题面出现"后缀""最长公共前缀""本质不同子串""重复出现""字典序第 k 小"
13 //适用注意: 离线(建好后串不再变化); 在线加字符请用 SAM
14 struct SuffixArray {
15     int n;
16     string s;
17     vector<int> sa, rnk, height;
18     vector<int> lg; // 稀疏表用
19     vector<vector<int>> st; // height 的 RMQ, 用于任意两后缀求 LCP
20
21     SuffixArray() {}
22     SuffixArray(const string& _s) { build(_s); }
23
24     void build(const string& _s) {
25         s = _s; n = s.size();
26         build_sa();
27         build_height();
28         build_rmq();
29     }
30
31     void build_sa() {
32         sa.resize(n); rnk.resize(n);
33         vector<int> oldrnk(n), cnt(max(n, 256));
34         // 初始: 按单个字符计数排序, 得到第一个 rank
35         for (int i = 0; i < n; i++) cnt[(unsigned char)s[i]]++;
36         for (int i = 1; i < (int)cnt.size(); i++) cnt[i] += cnt[i-1];
37         for (int i = n-1; i >= 0; i--) sa[--cnt[(unsigned char)s[i]]] = i;
38         rnk[sa[0]] = 0;
39         for (int i = 1; i < n; i++)
40             rnk[sa[i]] = rnk[sa[i-1]] + (s[sa[i]] != s[sa[i-1]]);
41         vector<int> sa2(n);
42         for (int w = 1; w < n; w <= 1) {
43             // 按第二关键字排序: 当前 sa 已按 rank 有序, 整体平移 w 即得到第二关键字序
44             int m = 0;

```

```

45         for (int i = n - w; i < n; i++) sa2[m++] = i;          // 第二关键字越界(视
// 作-1)的排最前
46         for (int i = 0; i < n; i++) if (sa[i] >= w) sa2[m++] = sa[i] - w;
47         // 按第一关键字(rank)计数排序, 稳定保留第二关键字的顺序
48         fill(cnt.begin(), cnt.begin() + n + 1, 0);
49         for (int i = 0; i < n; i++) cnt[rnk[sa2[i]]]++;
50         for (int i = 1; i <= n; i++) cnt[i] += cnt[i-1];
51         for (int i = n-1; i >= 0; i--) sa[--cnt[rnk[sa2[i]]]] = sa2[i];
52         // 更新 rank
53         oldrnk = rnk;
54         rnk[sa[0]] = 0;
55         for (int i = 1; i < n; i++) {
56             int a = sa[i], b = sa[i-1];
57             int k2a = a + w < n ? oldrnk[a + w] : -1;
58             int k2b = b + w < n ? oldrnk[b + w] : -1;
59             rnk[a] = rnk[b] + (oldrnk[a] != oldrnk[b] || k2a != k2b);
60         }
61         if (rnk[sa[n-1]] == n-1) break;      // rank 全不同, 提前结束
62     }
63 }
64
65 void build_height() {
66     height.assign(n, 0);
67     int k = 0;
68     for (int i = 0; i < n; i++) {
69         if (rnk[i] == 0) { k = 0; continue; } // 排名第一的没有前驱
70         int j = sa[rnk[i] - 1];
71         while (i+k < n && j+k < n && s[i+k] == s[j+k]) k++;
72         height[rnk[i]] = k;
73         if (k) k--;                          // 删首字符后 LCP 至少 k-1
74     }
75 }
76
77 void build_rmq() {
78     lg.assign(n + 1, 0);
79     for (int i = 2; i <= n; i++) lg[i] = lg[i/2] + 1;
80     int k = lg[n] + 1;
81     st.assign(k, vector<int>(n + 1));
82     for (int i = 0; i <= n; i++) st[0][i] = height[i];
83     for (int k = 1; k < k; k++)
84         for (int i = 0; i + (1 << k) <= n + 1; i++)
85             st[k][i] = min(st[k-1][i], st[k-1][i + (1 << (k-1))]);
86 }
87
88 // 任意两个后缀 s[x..] 与 s[y..] 的最长公共前缀长度
89 int lcp(int x, int y) {
90     if (x == y) return n - x;
91     int a = rnk[x], b = rnk[y];
92     if (a > b) swap(a, b);
93     int len = b - a;                          // 区间 [a+1, b] 长度, 查 height 最小值
94     int k = lg[len];
95     return min(st[k][a+1], st[k][b - (1 << k) + 1]);
96 }
97 };

```

后缀自动机

```

1 //后缀自动机模板 SAM (每个状态 = 一个 endpos 等价类)
2 //状态数不超过 2n, next 数组用 0 表示"无转移"(与根节点 0 复用, 注意根不能是转移目标)
3 //

```

```

4 //SAM 常用来解决什么:
5 // - 不同子串数 / 本质不同子串第 k 小 (O(n), 见 count_distinct)
6 // - 每个子串的出现次数 (endpos 大小, 见 count_endpos / occur)
7 // - 两串最长公共子串 (对 A 建 SAM 在 B 上跑)
8 // - 在线: 末尾加一个字符后新增的不同子串数 = len[last]-len[link[last]] (SA 做不到在线)
9 // - 进阶: 维护每个状态的 endpos 集合(线段树合并)可查子串出现的具体位置/区间
10 //判别信号: 需要按"出现次数/位置集合"统计任意子串; 数据范围 n 到 1e6 也能承受 O(n·26)
11 //适用注意: 本质是"子串自动机", 问"子串出现情况"用它; 问"后缀两两之间"用后缀树/后缀数组
12 struct SAM {
13     struct State {
14         int len, link;           // len: 该状态代表的最长子串长度; link: 后缀链接(fail)
15         int next[26];
16         void init() { len = link = 0; memset(next, 0, sizeof(next)); }
17     };
18     vector<State> st;
19     vector<int> isclone;         // 标记克隆点(统计 endpos 时非克隆点初值为 1)
20     int tot, last;
21
22     void init(int maxlen) {           // maxlen: 串的最大长度
23         int M = 2 * maxlen + 5;       // SAM 状态数上界 2n
24         st.assign(M, State());
25         isclone.assign(M, 0);
26         for (int i = 0; i < M; i++) st[i].init();
27         st[0].len = 0; st[0].link = -1;
28         tot = last = 0;
29     }
30
31     void extend(int c) {
32         int cur = ++tot;
33         st[cur].len = st[last].len + 1;
34         int p = last;
35         while (p != -1 && !st[p].next[c]) { st[p].next[c] = cur; p = st[p].link; }
36         if (p == -1) st[cur].link = 0; // 一路走到根都没有该转移, 直接连
根
37         else {
38             int q = st[p].next[c];
39             if (st[p].len + 1 == st[q].len) st[cur].link = q; // 不用分裂, 直接挂
40             else {
41                 int clone = ++tot;           // 需要分裂出克隆点
42                 isclone[clone] = 1;
43                 st[clone] = st[q];
44                 st[clone].len = st[p].len + 1;
45                 while (p != -1 && st[p].next[c] == q) { st[p].next[c] = clone; p =
st[p].link; }
46                 st[q].link = st[cur].link = clone;
47             }
48         }
49         last = cur;
50     }
51
52     void build(const string& s) {
53         for (char ch : s) extend(ch - 'a');
54     }
55
56     // 不同子串数 =  $\sum(\text{len}[v] - \text{len}[\text{link}[v]])$ 
57     long long count_distinct() {
58         long long res = 0;
59         for (int v = 1; v <= tot; v++) res += st[v].len - st[st[v].link].len;
60         return res;
61     }
62 }

```

```

63 // 统计每个状态 endpos 大小，即其代表的所有子串的出现次数
64 vector<int> cnt;
65 void count_endpos() {
66     int sz = tot + 1;
67     cnt.assign(sz, 0);
68     vector<int> bucket(sz, 0), order(sz);
69     for (int v = 0; v < sz; v++) if (st[v].len) bucket[st[v].len]++;
70     for (int i = 1; i < sz; i++) bucket[i] += bucket[i-1];
71     for (int v = sz-1; v >= 0; v--) if (st[v].len) order[--bucket[st[v].len]]
= v;
72     // 非克隆点初值 1(代表它是某个前缀的终点); 克隆点初值 0
73     for (int v = 0; v <= tot; v++) if (v && !isclone[v]) cnt[v] = 1;
74     // 按 len 降序就是拓扑序，从叶子往根累加
75     for (int i = sz-1; i > 0; i--) {
76         int v = order[i];
77         if (st[v].link > 0) cnt[st[v].link] += cnt[v];
78     }
79 }
80
81 // 模式串 t 在原串中的出现次数 (需先调用 count_endpos)
82 int occur(const string& t) {
83     int u = 0;
84     for (char ch : t) {
85         u = st[u].next[ch - 'a'];
86         if (!u) return 0;
87     }
88     return cnt[u];
89 }
90 };

```

后缀树

```

1 //后缀树模板：用"反串的 SAM 的 link 树"实现
2 //定理：S 的后缀树 = SAM(反转后的 S) 的 link(后缀链接)树
3 // - 每个 SAM 状态 = 后缀树一个节点，串深度(根到该节点路径上的字符数) = len[v]
4 // - 边 link[v] -> v 的标签长度 = len[v] - len[link[v]]
5 // - 后缀 s[p..] 对应的节点 = 处理反串第 n-1-p 个字符后的 last 状态 (记录在 leaf[p])
6 // - 不加哨兵时：若某后缀是另一后缀的前缀(如 "a" 是 "aa" 的前缀)，它对应的是内部节点，属正常
现象
7 // - 两个后缀的 LCP = 对应节点在 link 树上的 LCA 的 len
8 //
9 //后缀树常用来解决什么：
10 // - 批量查询任意两后缀的 LCP (预处理后单次 O(log n)，见 lcp)
11 // - 后缀树上 DP：子树"终点标记"之和 = 某前缀的出现次数；出现次数>=K 的最长子串
12 // - 本质不同子串数 / 各节点对应的公共前缀长度 (见 count_distinct)
13 // - 树上差分、可持久化等树技巧可以直接搬过来用
14 //判别信号：需要"后缀与后缀之间的关系"(两两 LCP、作为树处理)，或要做树形 DP
15 //与 SAM 的取舍：只需要子串出现统计时直接 SAM 更省；需要后缀间关系/树形态时用后缀树
16 struct SuffixTree {
17     struct State {
18         int len, link, next[26];
19         void init() { len = link = 0; memset(next, 0, sizeof(next)); }
20     };
21     vector<State> st;
22     int tot, last;
23     vector<int> leaf; // leaf[p]: 后缀 s[p..] 对应的树节点
24     vector<vector<int>> g; // link 树邻接表 (g[link[v]].push_back(v))
25     vector<int> dep; // 树深度(边数)，供倍增跳 LCA
26     vector<int> up[20]; // 倍增表
27

```

```

28 void init(int maxlen) {
29     int M = 2 * maxlen + 5;
30     st.assign(M, State());
31     for (int i = 0; i < M; i++) st[i].init();
32     st[0].len = 0; st[0].link = -1;
33     tot = last = 0;
34 }
35
36 void extend(int c) {
37     int cur = ++tot;
38     st[cur].len = st[last].len + 1;
39     int p = last;
40     while (p != -1 && !st[p].next[c]) { st[p].next[c] = cur; p = st[p].link;
41 }
42     if (p == -1) st[cur].link = 0;
43     else {
44         int q = st[p].next[c];
45         if (st[p].len + 1 == st[q].len) st[cur].link = q;
46         else {
47             int clone = ++tot;
48             st[clone] = st[q];
49             st[clone].len = st[p].len + 1;
50             while (p != -1 && st[p].next[c] == q) { st[p].next[c] = clone; p
51 = st[p].link; }
52             st[q].link = st[cur].link = clone;
53         }
54     }
55     last = cur;
56 }
57
58 void build(const string& s) {
59     int n = s.size();
60     init(n);
61     leaf.assign(n, 0);
62     for (int i = 0; i < n; i++) { // 对反串建 SAM
63         extend(s[n-1-i] - 'a'); // 反串第 i 个字符 = 原串的 s[n-1-
64 i]
65         leaf[n-1-i] = last; // 反串前缀长度 i+1 <-> 原串后缀起点
66 n-1-i
67     }
68     g.assign(tot + 1, {});
69     for (int v = 1; v <= tot; v++) g[st[v].link].push_back(v); // 建 link 树
70     // 迭代 DFS 预处理倍增表 (link 树可能退化成链, 递归会爆栈)
71     dep.assign(tot + 1, 0);
72     for (int k = 0; k < 20; k++) up[k].assign(tot + 1, 0);
73     vector<int> stk{0};
74     while (!stk.empty()) {
75         int u = stk.back(); stk.pop_back();
76         for (int v : g[u]) {
77             dep[v] = dep[u] + 1;
78             up[0][v] = u;
79             for (int k = 1; k < 20; k++) up[k][v] = up[k-1][up[k-1][v]];
80             stk.push_back(v);
81         }
82     }
83 }
84
85 int lca(int a, int b) {
86     if (dep[a] < dep[b]) swap(a, b);
87     int d = dep[a] - dep[b];
88     for (int k = 0; k < 20; k++) if (d >> k & 1) a = up[k][a];

```

```
85     if (a == b) return a;
86     for (int k = 19; k >= 0; k--)
87         if (up[k][a] != up[k][b]) { a = up[k][a]; b = up[k][b]; }
88     return up[0][a];
89 }
90
91 // 后缀 s[p..] 与 s[q..] 的最长公共前缀长度 = 叶子 LCA 的串深度
92 int lcp(int p, int q) { return st[lca(leaf[p], leaf[q])].len; }
93
94 // 不同子串数 =  $\sum$  每条边的标签长度
95 long long count_distinct() {
96     long long res = 0;
97     for (int v = 1; v <= tot; v++) res += st[v].len - st[st[v].link].len;
98     return res;
99 }
100 };
```